

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#02)

Implementation of Combinational Circuits using Verilog Gate & Dataflow Level Modeling

Student Name: Fatima Tariq Roll No: 24i-6557 Section: CE-A

Date performed: 31. August, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1: Write Verilog Code for Half Adder and Full adder using Gate level and Dataflow level modeling. Simulate it by writing testbench.

Description:

In this task a Half adder and a full adder were implemented on Modelsim using both Gate Level and data flow level HDL and both were also implemented on Logisim to validate results.

Code:

Half Adder Gate Level:

```
module task01(sum,carry,a,b);
input a;
input b;
output sum;
output carry;
xor x1(sum,a,b);
and a1(carry,a,b);
endmodule
```

```
module testbench_ha();
reg a;
reg b;
wire sum,carry;

task01 hht(sum,carry,a,b);
initial
begin
a=0;b=0;
#50 a=1;b=0;
#50 a=0;b=1;
#50 a=1;b=1;
end
endmodule
```

Half Adder Data Flow Level:

```
module task01(sum,carry,a,b);
input a;
input b;
output sum;
output carry;
assign sum=a^b;
assign carry=a&b;
endmodule
```

```
module testbench_ha();
reg a;
```

```
reg b;
wire sum,carry;

task01 hht(sum,carry,a,b);
initial
begin
a=0;b=0;
#50 a=1;b=0;
#50 a=0;b=1;
#50 a=1;b=1;
end
endmodule
```

Full Adder Gate Level:

```
module ha(sum, carry, a, b);
input a, b;
output sum, carry;

xor x1(sum, a, b);
and a1(carry, a, b);
endmodule

module task01(sum, cin, cout, a, b);
input a, b, cin;
output sum, cout;
wire sum1, carry1, carry2;

ha h1(sum1, carry1, a, b);
ha h2(sum, carry2, sum1, cin);
or o1(cout, carry1, carry2);
endmodule

module testbench_fa();
reg a, b, c;
wire sum, cout;

task01 fa(sum, c, cout, a, b);

initial begin
a=0; b=0; c=0;
#50 a=0; b=0; c=1;
#50 a=0; b=1; c=0;
#50 a=0; b=1; c=1;
#50 a=1; b=0; c=0;
#50 a=1; b=0; c=1;
#50 a=1; b=1; c=0;
#50 a=1; b=1; c=1;

end
endmodule
```

Full Adder Data flow Level:

```
module ha(sum, carry, a, b);
```

```
input a, b;
output sum, carry;
assign sum=a^b;
assign carry=a&b;
endmodule

module task01(sum, cin, cout, a, b);
input a, b, cin;
output sum, cout;
wire sum1, carry1, carry2;

ha h1(sum1, carry1, a, b);
ha h2(sum, carry2, sum1, cin);
assign cout=carry1|carry2;
endmodule

module testbench_fa();
reg a, b, c;
wire sum, cout;

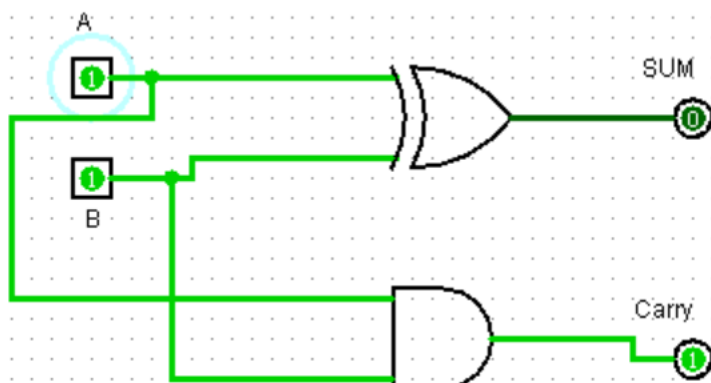
task01 fa(sum, c, cout, a, b);

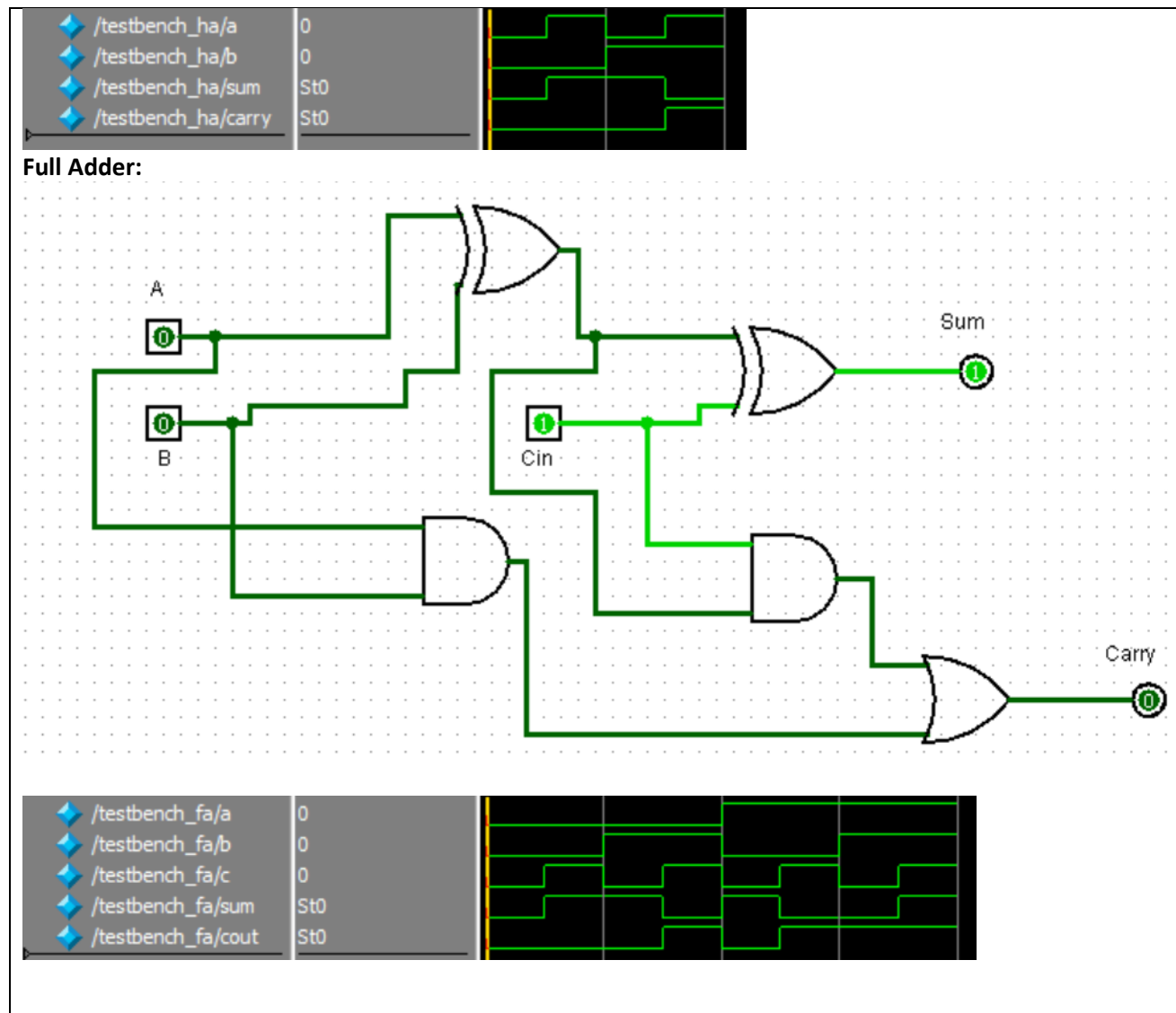
initial begin
a=0; b=0; c=0;
#50 a=0; b=0; c=1;
#50 a=0; b=1; c=0;
#50 a=0; b=1; c=1;
#50 a=1; b=0; c=0;
#50 a=1; b=0; c=1;
#50 a=1; b=1; c=0;
#50 a=1; b=1; c=1;

end
endmodule
```

Output:

Half adder:





Task 2: Write Verilog code of 3x8 Decoder using Gate level and Dataflow level modeling
simulate the results by writing testbench.

Description:

In this task a 3x8 decoder was implemented on both modelsim and Logisim and on model sim it was implemented using data flow and gate level HDL (Verilog) language and in Logisim it was also implemented using basic logic gates.

Code:

3X8 Decoder Gate Level:

```
module task02(a,b,c,D0,D1,D2,D3,D4,D5,D6,D7);
input a,b,c;
output D0,D1,D2,D3,D4,D5,D6,D7;
wire anot,bnot,cnot;
not n1(anot,a);
not n2(bnot,b);
not n3(cnot,c);
and a1(D0,anot,bnot,cnot); //C'B'A'
and a2(D1,anot,bnot,c); //C'B'A
and a3(D2,anot,b,cnot); //C'BA'
and a4(D3,anot,b,c); //C'BA
and a5(D4,a,bnot,cnot); //CB'A'
and a6(D5,a,bnot,c); //CB'A
and a7(D6,a,b,cnot); //CBA'
and a8(D7,a,b,c); //CBA
endmodule
```

```
module testbench_decoder();
reg a,b,c;
wire D0,D1,D2,D3,D4,D5,D6,D7;
```

```
task02 decodde(a,b,c,D0,D1,D2,D3,D4,D5,D6,D7);
initial
begin
a=0;b=0;c=0;
#50 a=0;b=0;c=1;
#50 a=0;b=1;c=0;
#50 a=0;b=1;c=1;
#50 a=1;b=0;c=0;
#50 a=1;b=0;c=1;
#50 a=1;b=1;c=0;
#50 a=1;b=1;c=1;
end
endmodule
```

3X8 Decoder Data Flow Level:

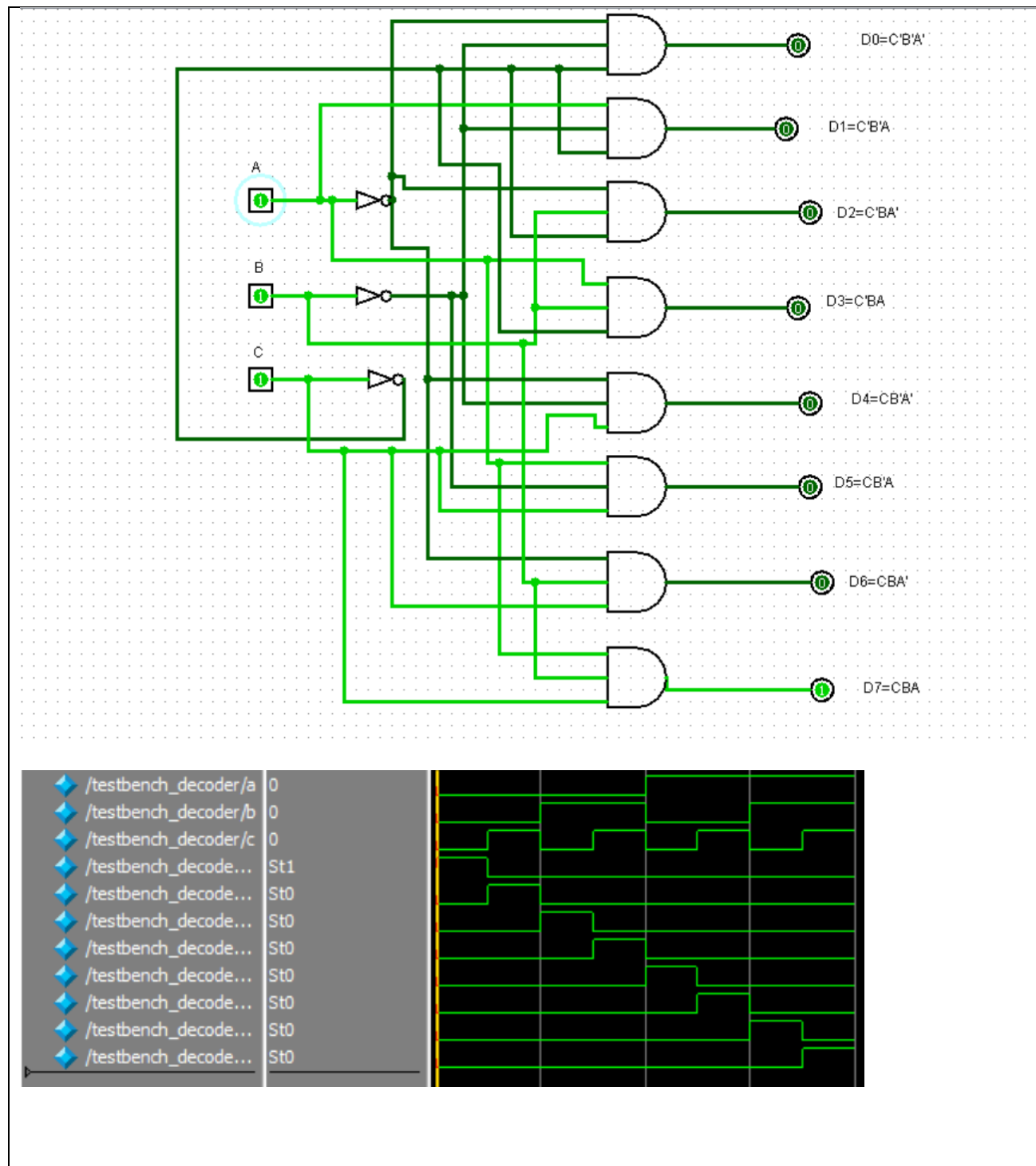
```
module task02(a,b,c,D0,D1,D2,D3,D4,D5,D6,D7);
input a,b,c;
output D0,D1,D2,D3,D4,D5,D6,D7;
wire anot,bnot,cnot;
assign anot=~a;
assign bnot=~b;
assign cnot=~c;
assign D0=anot&bnot&cnot;
assign D1=anot&bnot&c;
assign D2=anot&b&cnot;
assign D3=anot&b&c;
assign D4=a&bnot&cnot;
assign D5=a&bnot&c;
assign D6=a&b&cnot;
assign D7=a&b&c;
endmodule

module testbench_decoder();
reg a,b,c;
wire D0,D1,D2,D3,D4,D5,D6,D7;

task02 decodde(a,b,c,D0,D1,D2,D3,D4,D5,D6,D7);
initial
begin
a=0;b=0;c=0;
#50 a=0;b=0;c=1;
#50 a=0;b=1;c=0;
#50 a=0;b=1;c=1;
#50 a=1;b=0;c=0;
#50 a=1;b=0;c=1;
#50 a=1;b=1;c=0;
#50 a=1;b=1;c=1;
end
endmodule
```

Output:

3x8 Decoder:



Task 3: Write Verilog code of 8x1 multiplexer using Gate level and Dataflow modeling. Simulate the results by writing testbench.

Description:

In this task a 8x1 multiplexer was implemented on both modelsim and Logisim and on model sim it was implemented using data flow and gate level HDL (Verilog) language and in Logisim it was also implemented using basic logic gates.

Code:

8x1 Multiplexer Gate Level:

```
module task03(S2,S1,S0,A0,A1,A2,A3,A4,A5,A6,A7,y);
input S2,S1,S0,A0,A1,A2,A3,A4,A5,A6,A7;
output y;
wire s2not,s1not,s0not,a0,a1,a2,a3,a4,a5,a6,a7;
not n1(s2not,S2);
not n2(s1not,S1);
not n3(s0not,S0);
and aa0(a0,s2not,s1not,s0not,A0); //A0S2'S1'S0'
and aa1(a1,s2not,s1not,S0,A1); //A1S2'S1'S0
and aa2(a2,s2not,S1,s0not,A2); //A2S2'S1S0'
and aa3(a3,s2not,S1,S0,A3); //A3S2'S1S0
and aa4(a4,S2,s1not,s0not,A4); //A4S2S1'S0'
and aa5(a5,S2,s1not,S0,A5); //A5S2S1'S0
and aa6(a6,S2,S1,s0not,A6); //A6S2S1S0'
and aa7(a7,S2,S1,S0,A7); //A7S2S1S0
or o1(y,a0,a1,a2,a3,a4,a5,a6,a7);
endmodule
```

```
module testbench_multiplexer();
reg a0,a1,a2,a3,a4,a5,a6,a7,s0,s1,s2;
wire y;

task03 tts(s2,s1,s0,a0,a1,a2,a3,a4,a5,a6,a7,y);
initial
begin
a0=1; a1=1; a2=1; a3=1; a4=1; a5=1; a6=1; a7=1;
s2=0; s1=0; s0=0;
#50 s2=0; s1=0; s0=1;
#50 s2=0; s1=1; s0=0;
#50 s2=0; s1=1; s0=1;
#50 s2=1; s1=0; s0=0;
#50 s2=1; s1=0; s0=1;
#50 s2=1; s1=1; s0=0;
#50 s2=1; s1=1; s0=1;

#50;
a0=0; a1=0; a2=0; a3=0; a4=0; a5=0; a6=0; a7=0;
s2=0; s1=0; s0=0;
#50 s2=0; s1=0; s0=1;
#50 s2=0; s1=1; s0=0;
#50 s2=0; s1=1; s0=1;
```

```
#50 s2=1; s1=0; s0=0;
#50 s2=1; s1=0; s0=1;
#50 s2=1; s1=1; s0=0;
#50 s2=1; s1=1; s0=1;

end
endmodule
3X8 Decoder Data Flow Level:
module task03(S2,S1,S0,A0,A1,A2,A3,A4,A5,A6,A7,y);
input S2,S1,S0,A0,A1,A2,A3,A4,A5,A6,A7;
output y;
wire s2not,s1not,s0not,a0,a1,a2,a3,a4,a5,a6,a7;

assign s2not = ~S2;
assign s1not = ~S1;
assign s0not = ~S0;

assign a0 = s2not & s1not & s0not & A0;
assign a1 = s2not & s1not & S0 & A1;
assign a2 = s2not & S1 & s0not & A2;
assign a3 = s2not & S1 & S0 & A3;
assign a4 = S2 & s1not & s0not & A4;
assign a5 = S2 & s1not & S0 & A5;
assign a6 = S2 & S1 & s0not & A6;
assign a7 = S2 & S1 & S0 & A7;

assign y = a0 | a1 | a2 | a3 | a4 | a5 | a6 | a7;
endmodule

module testbench_multiplexer();
reg a0,a1,a2,a3,a4,a5,a6,a7,s0,s1,s2;
wire y;

task03 tts(s2,s1,s0,a0,a1,a2,a3,a4,a5,a6,a7,y);
initial
begin
a0=1; a1=1; a2=1; a3=1; a4=1; a5=1; a6=1; a7=1;
s2=0; s1=0; s0=0;
#50 s2=0; s1=0; s0=1;
#50 s2=0; s1=1; s0=0;
#50 s2=0; s1=1; s0=1;
#50 s2=1; s1=0; s0=0;
#50 s2=1; s1=0; s0=1;
#50 s2=1; s1=1; s0=0;
#50 s2=1; s1=1; s0=1;

#50;
a0=0; a1=0; a2=0; a3=0; a4=0; a5=0; a6=0; a7=0;
s2=0; s1=0; s0=0;
#50 s2=0; s1=0; s0=1;
#50 s2=0; s1=1; s0=0;
#50 s2=0; s1=1; s0=1;
#50 s2=1; s1=0; s0=0;
#50 s2=1; s1=0; s0=1;
```

```
#50 s2=1; s1=0; s0=1;  
#50 s2=1; s1=1; s0=0;  
#50 s2=1; s1=1; s0=1;
```

```
end  
endmodule
```

Output:

8x1 Multiplexer:

